

Eccezioni

Eccezioni

Come gestire situazioni impreviste con try, catch e finally.

Cosa è un'eccezione

Un'eccezione è un problema che avviene mentre il programma è in esecuzione.

Esempi:

- un file non esiste
- l'utente inserisce testo al posto di un numero
- un array viene letto fuori dai suoi limiti

Java usa le eccezioni per segnalare questi casi.

try e catch

Con `try catch` puoi provare un'operazione e gestire l'errore se succede.

```
try {  
    int numero = Integer.parseInt("abc");  
    System.out.println(numero);  
} catch (NumberFormatException e) {  
    System.out.println("Non è un numero valido.");  
}
```

Output:

```
Non è un numero valido.
```

`Integer.parseInt("abc")` fallisce perché `"abc"` non è un numero.

Un esempio con input

```
public class LetturaNumero {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Numero: ");
        String testo = scanner.nextLine();

        try {
            int numero = Integer.parseInt(testo);
            System.out.println("Doppio: " + (numero * 2));
        } catch (NumberFormatException e) {
            System.out.println("Devi inserire un numero intero.");
        }

        scanner.close();
    }
}
```

Se l'utente scrive `12`, il programma calcola il doppio.

Se scrive `ciao`, il programma mostra un messaggio chiaro invece di interrompersi.

finally

`finally` contiene codice che viene eseguito comunque, sia se tutto va bene sia se c'è un errore.

```
try {
    System.out.println("Provo a fare qualcosa");
} catch (Exception e) {
    System.out.println("Qualcosa è andato storto");
} finally {
    System.out.println("Questa riga viene eseguita sempre");
}
```

È utile per chiudere risorse, come file o connessioni.

Eccezioni controllate e non controllate

Java distingue due famiglie importanti.

Le **eccezioni controllate** devono essere gestite o dichiarate. Un esempio comune è `IOException`, usata quando lavori con file.

Le **eccezioni non controllate** possono avvenire a runtime e Java non ti obbliga sempre a gestirle. Esempi:

- `NumberFormatException`
- `ArrayIndexOutOfBoundsException`
- `NullPointerException`

Per iniziare non devi memorizzare tutte le categorie. Ti basta sapere che alcune operazioni, come i file, obbligano a pensare agli errori.

Non catturare tutto senza motivo

Potresti scrivere:

```
catch (Exception e) {  
    System.out.println("Errore");  
}
```

Funziona, ma è molto generico.

Quando puoi, cattura l'eccezione specifica:

```
catch (NumberFormatException e) {  
    System.out.println("Numero non valido.");  
}
```

Il messaggio è più chiaro e il codice è più facile da controllare.

Regola pratica

Usa le eccezioni per gestire problemi che possono capitare anche in un programma scritto bene: input sbagliato, file mancanti, dati non validi.

Non usarle per nascondere errori che dovresti correggere nel codice.